Modélisation et Résolution Numérique de l'Équation de la Chaleur

Formulation Théorique et Implémentation Orientée Objet (C++)

Projet de Simulation Thermique

25 février 2026

1. Introduction

Ce rapport présente la modélisation mathématique et la discrétisation numérique de la conduction thermique. L'objectif est de transformer une loi physique continue en un algorithme discret permettant de simuler l'évolution temporelle de la température T dans un domaine bidimensionnel (2D), avec extension naturelle en 3D.

D'un point de vue informatique, ce projet consiste à implémenter un **solveur itératif** basé sur la méthode des différences finies, structuré autour d'une architecture logicielle robuste présentée en fin de document.

2. Définition de la densité de flux thermique j_{th}

La variable j_{th} représente la **densité de flux thermique**. C'est une grandeur vectorielle (réduite à une composante scalaire dans notre dérivation 1D) qui mesure l'énergie thermique traversant une surface par unité de temps.

— **Interprétation physique :** Elle caractérise la "vitesse" et la "direction" du transfert de chaleur. Plus le matériau est conducteur ou plus l'écart de température est brutal, plus j_{th} est élevé.

— **Loi de Fourier :** Elle est régie par la relation :

$$j_{th} = -\lambda \frac{\partial T}{\partial x} \tag{1}$$

Le signe **négatif** est fondamental : il indique que la chaleur s'écoule naturellement des zones chaudes vers les zones froides (à l'opposé du gradient de température).

— **Unité :** Elle s'exprime en **Watts par mètre carré** ($W \cdot m^{-2}$).

2.1 Le vecteur densité de flux thermique $\vec{j}_{th}$ en 2D et 3D

Dans un espace à plusieurs dimensions, la chaleur ne se déplace pas seulement de gauche à droite, mais dans toutes les directions de l'espace. Le flux thermique devient alors un **champ vectoriel**.

— **Définition vectorielle :** La densité de flux thermique $\vec{j}_{th}$ représente la puissance thermique traversant une unité de surface orthogonale au flux.

— **Loi de Fourier généralisée :** Le lien entre le flux et la température est donné par l'opérateur **gradient** ($\vec{\nabla}$) :

$$\vec{j}_{th} = -\lambda \vec{\nabla} T \tag{2}$$

Décomposition par dimension

Selon la dimension de la simulation choisie dans le logiciel, le vecteur se décompose comme suit :

En 2D (Plan xy) : Le flux a deux composantes. La chaleur peut se propager latéralement et verticalement.

$$\vec{j}_{th} = \begin{pmatrix} j_x \\ j_y \end{pmatrix} = \begin{pmatrix} -\lambda \frac{\partial T}{\partial x} \\ -\lambda \frac{\partial T}{\partial y} \end{pmatrix} \tag{3}$$

En 3D (Espace xyz) : C'est le cas le plus complet pour un objet réel. Le vecteur possède trois composantes.

$$\vec{j}_{th} = \begin{pmatrix} j_x \\ j_y \\ j_z \end{pmatrix} = \begin{pmatrix} -\lambda \frac{\partial T}{\partial x} \\ -\lambda \frac{\partial T}{\partial y} \\ -\lambda \frac{\partial T}{\partial z} \end{pmatrix} \tag{4}$$

Interprétation

- **Direction du transfert :** Le vecteur pointe toujours vers la direction où la température diminue le plus rapidement (du chaud vers le froid).
- **Intensité :** La norme du vecteur $\left\|\vec{j}_{th}\right\|$ indique la quantité de Joules qui transitent par seconde et par m^2.
- **Application aux bords (Neumann) :** Lorsqu'on définit une condition de Neumann sur une face (ex : la face NORD), on impose en réalité la composante du vecteur flux normale à cette face (j_y pour le nord).

3. Dérivation de l'Équation de la Chaleur

La formulation repose sur le premier principe de la thermodynamique appliqué à un volume de contrôle élémentaire.

3.1 Bilan énergétique

Le premier principe s'écrit :

$$dU = \delta Q + \delta W \tag{5}$$

Sous les hypothèses :
- **Milieu immobile** (pas de transport de matière)
- **Volume constant** (pas de dilatation thermique prise en compte)
- **Absence de travail mécanique**

On obtient :

$$dU = \delta Q \tag{6}$$

3.2 Flux thermique

Considérons un élément de volume $S\,dx$. La chaleur nette reçue est la différence des flux conductifs entre l'entrée et la sortie du volume :

$$\delta Q = [j_{th}(x,t) - j_{th}(x+dx,t)]\, S\, dt \tag{7}$$

Par développement limité au premier ordre :

$$\delta Q = -\frac{\partial j_{th}}{\partial x} S\, dx\, dt \tag{8}$$

3.3 Relation énergie-température

Pour un milieu de masse volumique ρ et capacité thermique massique c :

$$dU = \rho c S dx \, dT \tag{9}$$

Comme la température varie dans le temps $dT = \frac{\partial T}{\partial t} dt$:

$$dU = \rho c S dx \frac{\partial T}{\partial t} dt \tag{10}$$

3.4 Équation locale de conservation

En identifiant les deux expressions de l'énergie interne dU :

$$\rho c \frac{\partial T}{\partial t} = -\frac{\partial j_{th}}{\partial x} \tag{11}$$

Avec la **loi de Fourier**, qui lie le flux au gradient de température :

$$j_{th} = -\lambda \frac{\partial T}{\partial x} \tag{12}$$

On obtient l'équation finale :

$$\rho c \frac{\partial T}{\partial t} = \lambda \frac{\partial^2 T}{\partial x^2} \tag{13}$$

D'où l'équation de diffusion :

$$\boxed{\frac{\partial T}{\partial t} = D \frac{\partial^2 T}{\partial x^2}} \tag{14}$$

En présence d'une source volumique de chaleur σ (ex : effet Joule, réaction chimique) :

$$\frac{\partial T}{\partial t} = D \frac{\partial^2 T}{\partial x^2} + \frac{\sigma}{\rho c} \tag{15}$$

3.5 Nomenclature et Définitions

Pour faciliter l'implémentation logicielle, voici le récapitulatif des variables utilisées :

Symbole	**Nom**	**Unité (SI)**	**Classe C++**
T	Température	K (Kelvin)	`Grid`
t	Temps	s (seconde)	`Simulation`
λ	Conductivité thermique	$W \cdot m^{-1} \cdot K^{-1}$	`Material`
ρ	Masse volumique	$kg \cdot m^{-3}$	`Material`
c	Capacité thermique massique	$J \cdot kg^{-1} \cdot K^{-1}$	`Material`
D	Diffusivité thermique	$m^2 \cdot s^{-1}$	`Material`
j_{th}	Densité de flux thermique	$W \cdot m^{-2}$	-
σ	Source de chaleur volumique	$W \cdot m^{-3}$	`Simulation`

— **La conductivité** (λ) exprime la facilité du matériau à transmettre la chaleur.
— **La diffusivité** (D) représente la vitesse de propagation de la chaleur à travers le matériau. C'est le paramètre clé du solveur.

4. Extension 2D et 3D

En dimension supérieure, l'équation devient :

$$\frac{\partial T}{\partial t} = D\Delta T + s \tag{16}$$

où $\Delta = \nabla^2$ est le Laplacien.

En 2D, le Laplacien s'exprime par :

$$\Delta T = \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \tag{17}$$

Note informatique : En 3D, il suffira d'ajouter le terme $\frac{\partial^2 T}{\partial z^2}$ à notre opérateur de calcul sans modifier la structure globale du solveur.

5. Discrétisation Spatiale : Méthode des Différences Finies

5.1 Maillage (Grid)

Le domaine est discrétisé en cellules régulières de pas Δx, Δy. La température continue devient une structure de données discrète (Matrice ou Tableau) :

$$T(x, y, t) \longrightarrow T_{i,j}^n \tag{18}$$

5.2 Approximation du Laplacien : Le Stencil

Pour calculer la variation spatiale, nous utilisons un développement de Taylor d'ordre 2. En informatique, on appelle cela un **stencil** (motif de voisinage). Pour la cellule (i, j), on regarde ses voisins directs :

$$\Delta T_{i,j} \approx \frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2} \tag{19}$$

6. Discrétisation Temporelle

6.1 Schéma d'Euler explicite

L'état futur de la grille $(n + 1)$ est calculé uniquement à partir des valeurs connues à l'instant présent (n) :

$$T_{i,j}^{n+1} = T_{i,j}^n + \Delta t \cdot D \left[\frac{T_{i+1,j}^n - 2T_{i,j}^n + T_{i-1,j}^n}{\Delta x^2} + \frac{T_{i,j+1}^n - 2T_{i,j}^n + T_{i,j-1}^n}{\Delta y^2} \right] \tag{20}$$

6.2 Condition de stabilité (CFL)

Pour un informaticien, cette condition est critique : si elle n'est pas respectée, les valeurs dans la mémoire vont diverger vers l'infini (NaN). La classe `Calculator` doit imposer :

$$\boxed{\Delta t \leq \frac{1}{2D\left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}\right)}} \tag{21}$$

7. Conditions aux Limites (Boundary Conditions)

Le code gère la mise à jour des bords de la `Grid` via trois types de conditions :

7.1 Dirichlet (Température imposée)

On fixe la valeur du bord à une constante : $T_{bord} = T_{fixe}$. Utile pour modéliser un contact avec une source de chaleur infinie.

7.2 Neumann (Flux imposé)

On impose le gradient de température : $-\lambda \frac{\partial T}{\partial n} = j_{impos}$. **Cas particulier :** Si le flux est nul, la paroi est dite "adiabatique" (isolation parfaite).

7.3 Robin (Convection)

Modélise l'échange avec un fluide ambiant :

$$-\lambda \frac{\partial T}{\partial n} = h_{conv}(T - T_{amb}) \tag{22}$$

Discrétisation sur une frontière Est :

$$T_{nx,j} = \frac{\lambda T_{nx-1,j} + h_{conv} \Delta x T_{amb}}{\lambda + h_{conv} \Delta x} \tag{23}$$

8. Architecture Logicielle et Implémentation POO

Le projet est découpé en classes selon le diagramme fourni, afin de séparer la physique de la gestion mémoire.

8.1 Rôle des classes

- **Grid** : Encapsule le `std::vector` de données. Gère l'indexage $(i, j) \to k$ et les dimensions spatiales.
- **Material** : Stocke les constantes physiques (ρ, c, λ) et calcule la diffusivité D.
- **Calculator** : Contient le cœur algorithmique (le stencil). C'est ici que l'équation discrète est résolue.
- **BoundaryCondition** : Interface abstraite permettant d'appliquer dynamiquement Dirichlet, Neumann ou Robin sur n'importe quel `Side` (North, South, East, West).
- **Simulation** : Orchestre la boucle temporelle et la mise à jour des données.